

RAbHIT: R Antibody Haplotype Inference Tool

Moriah Gidoni, Ayelet Peres, Gur Yaari

Last modified 2018-10-22

Contents

Introduction	1
Input	1
Running RAbHIT	2
References	8

Introduction

Analysis of antibody repertoires by high throughput sequencing is of major importance in understanding adaptive immune responses. Our knowledge of variations in the genomic loci encoding antibody genes is incomplete, mostly due to technical difficulties in aligning short reads to these highly repetitive loci. The partial knowledge results in conflicting *V-D-J* gene assignments between different algorithms, and biased genotype and haplotype inference. Previous studies have shown that haplotypes can be inferred by taking advantage of *IGHJ6* heterozygosity, observed in approximately one third of the population.

Here we provide a robust novel method for determining *V-D-J* haplotypes by adapting a Bayesian framework, **HaplotypeR**. Our method extends haplotype inference to *IGHD*- and *IGHV*-based analysis, thereby enabling inference of complex genetic events like deletions and copy number variations in the entire population. It calculates a Bayes factor, a number that indicates the certainty level of the inference, for each haplotyped gene.

More details can be found here:

Mosaic deletion patterns of the human antibody heavy chain gene locus as revealed by Bayesian haplotyping.

Input

HaplotypeR requires two main inputs:

1. Pre-processed antibody repertoire sequencing data with heterozygosity in at least one gene
2. Database of germline gene sequences

Antibody repertoire sequencing data is in a data frame format. Each row represents a unique observation and columns represent data about that observation. The names of the required columns are provided below along with a short description.

Column Name	Description
SUBJECT	Subject name
V_CALL	(Comma separated) name(s) of the nearest V allele(s) (IMGT format)

Column Name	Description
D_CALL	(Comma separated) name(s) of the nearest J allele(s)
J_CALL	(Comma separated) name(s) of the nearest J allele(s)

An example dataset is provided with the **rabhit** package. It contains unique naive b-cell sequences, from a single individual.

The database of germline sequences should be provided in FASTA format with sequences gapped according to the IMGT numbering scheme ([3]). IGHV alleles in the IMGT database (build 201408-4) are provided with this package. (object name)

```
library(rabhit)
# Load example sequence data and example germline database
data(sample_db, HVGERM, HDGERM)
```

Pre-processing of the data

One can use TIGGER functions to filter the data. . . . It is important to note which cell types/ exp. protocol. . . one representative from each clone (see SHAZAM. . .).

Running RAbHIT

The functions provided by this package can be used to perform any combination of the following:

1. Infer haplotype by anchor gene
2. Infer D/J single chromosome deletions by the V pooled approach
3. Infer double chromosome deletion by relative gene usage
4. Graphical output of the inferred haplotype
5. Graphical output of the inferred deletion

Infer haplotype by anchor gene

An individual's haplotype can be inferred using the functions **createFullHaplotype**. The function infers the haplotype based on the provided anchor gene. Using this function a contingency table is created for each gene, *from which strand is inferred for each allele*. The user can set the anchor gene for haplotyping as well as the column for which a haplotype should be inferred.

Prior to haplotyping, it is recommended to run TIGGER on the data, to detect new alleles and construct a genotype [link to docs].

```
# Inferred haplotype summary table
haplo_db <- createFullHaplotype(sample_db, toHap_col=c("V_CALL", "D_CALL"),
hapBy_col="J_CALL", hapBy="IGHJ6", toHap_GERM=c(HVGERM, HDGERM))

## In sample S32, 6475 sequences were removed due to multiple assignments,
## 40108 sequences left.

head(haplo_db)
```

##	SUBJECT	GENE	MinorFraction	DoubleAllele	IGHJ6_02	IGHJ6_03	ALLELES
## 1	S32	IGHV3-7	1	0	01	01	01
## 2	S32	IGHV3-21	1	0	01	01	01
## 3	S32	IGHV1-69	0.161	1	02	01,06	01,02,06
## 4	S32	IGHV3-30	1	0	18	18	18
## 5	S32	IGHV3-64D	1	0	06	Del	06
## 6	S32	IGHV1-8	1	0	Del	01	01

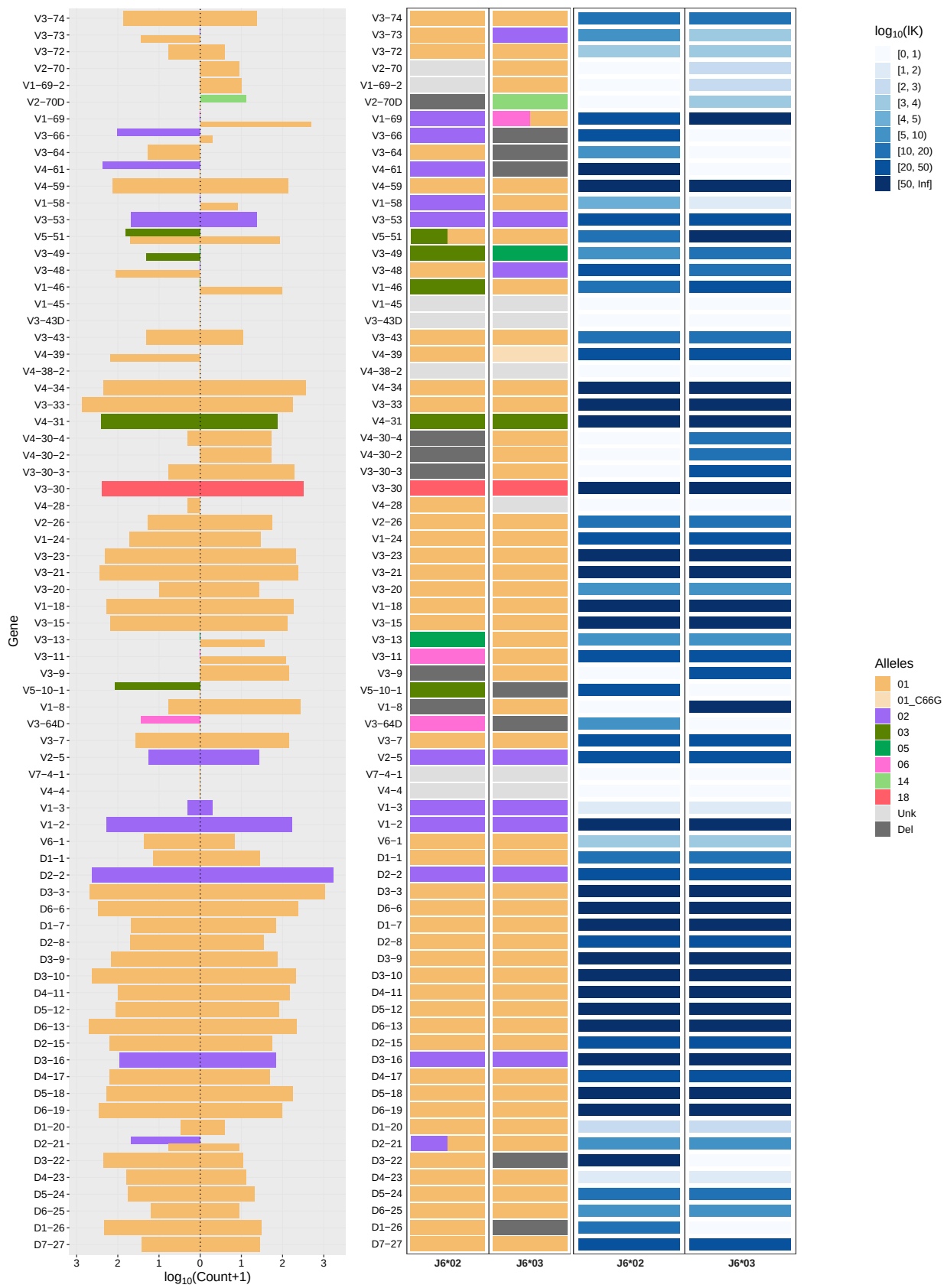
##	PRIORS_ROW	PRIORS_COL	COUNTS1	MP1	K1
## 1	0.48,0.52	1	36,140	-11.590764430191	21.4730927217493
## 2	0.48,0.52	1	281,237	-0.365273971780928	320.493783339993
## 3	0.48,0.52	0.52,0.19,0.30	0,484	0.61480351663325	136.343448274159
## 4	0.48,0.52	1	244,324	-0.0986982802780455	321.574048607258
## 5	0.48,0.52	1	27,0	1.3316304859018	8.4332169128303
## 6	0.48,0.52	1	5,273	1.3598271537726	68.4445656255678

##	ND1	COUNTS2	MP2	K2	ND2	COUNTS3
## 1	0	<NA>	<NA>	<NA>	<NA>	<NA>
## 2	0	<NA>	<NA>	<NA>	<NA>	<NA>
## 3	0	146,0	1.54261282977404	45.601839602712	0	0,279
## 4	0	<NA>	<NA>	<NA>	<NA>	<NA>
## 5	0	<NA>	<NA>	<NA>	<NA>	<NA>
## 6	0	<NA>	<NA>	<NA>	<NA>	<NA>

##	MP3	K3	ND3	COUNTS4	MP4	K4	ND4
## 1	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
## 2	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
## 3	1.25337339512454	78.5946736952282	0	<NA>	<NA>	<NA>	<NA>
## 4	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
## 5	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
## 6	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>

```
# Plot the haplotype
plotHaplotype(haplo_db)
```

S32



```

# Plot interactive haplotype plot
p <- plotHaplotype(haplo_db,html_output = T)
#save plot to html output
htmlwidgets::saveWidget(p, "haplotype.html",selfcontained = T)

```

Haplotype inference deletion heatmap

For a group of individuals, the deletions detected by the haplotyping process can be visualized with `deletionHeatmap`. The function create a heatmap of the deletion inferred and colors them by the certainty level (IK).

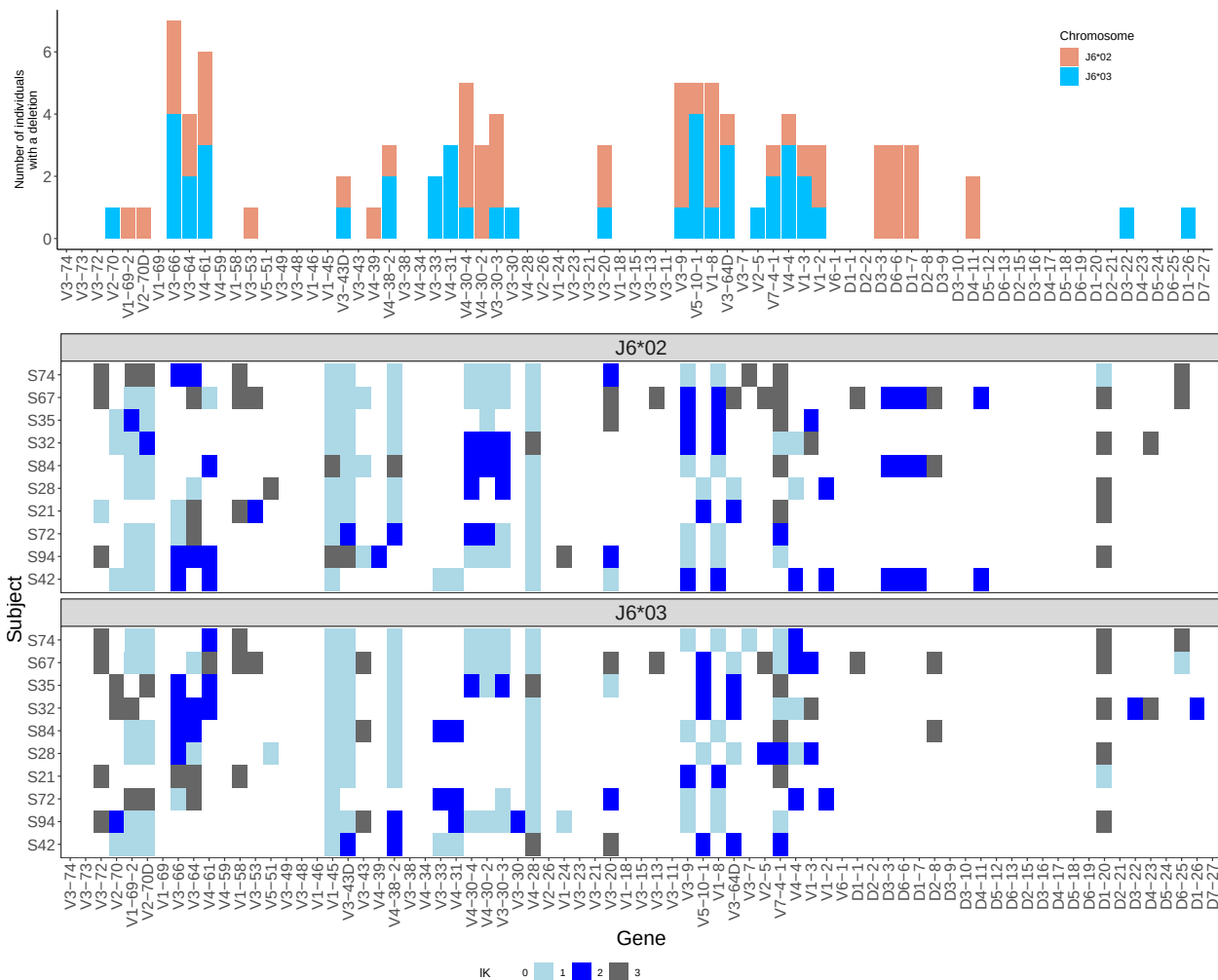
```

# Load example sequence data
data(samples_db)
# Inferred haplotype summary table for multiple subjects
haplo_db <- createFullHaplotype(samples_db,toHap_col=c("V_CALL","D_CALL"),
hapBy_col="J_CALL",hapBy="IGHJ6",toHap_GERM=c(HVGERM,HDGERM))

## In sample S42, 6066 sequnces were removed due to multiple assignments,
## 36776 sequences left.
## In sample S94, 6494 sequnces were removed due to multiple assignments,
## 29149 sequences left.
## In sample S72, 6082 sequnces were removed due to multiple assignments,
## 33301 sequences left.
## In sample S21, 6495 sequnces were removed due to multiple assignments,
## 29070 sequences left.
## In sample S28, 4569 sequnces were removed due to multiple assignments,
## 28225 sequences left.
## In sample S84, 9982 sequnces were removed due to multiple assignments,
## 43847 sequences left.
## In sample S32, 6475 sequnces were removed due to multiple assignments,
## 40108 sequences left.
## In sample S35, 5711 sequnces were removed due to multiple assignments,
## 32565 sequences left.
## In sample S67, 1390 sequnces were removed due to multiple assignments,
## 8559 sequences left.
## In sample S74, 4079 sequnces were removed due to multiple assignments,
## 21957 sequences left.

# plot deletion heatmap
deletionHeatmap(haplo_db)

```



Infering D/J single chromosome deletion by V pooled approach

Since V gene heterozygosity is extremely common, using V genes as anchors for haplotype inference could dramatically increase the number of people for which D haplotype can be inferred. However, reliable haplotype inference using V genes as anchors requires a much greater sequencing depth than haplotype inference using J6 gene as an anchor.

The `haplotper` package offers a solution to overcome the low number of sequences that connect a given V-D allele pair. The package function applies an aggregation approach, in which information from several V heterozygous genes can be combined to infer D gene deletions.

The `deletionsByVpooled` function uses the V pooled approach to detect single chromosomal deletion for D and J.

```
# Inferred deletion summary table
```

```
del_db <- deletionsByVpooled(samples_db)
```

```
## [1] "5093 sequences were removed due to multiple assignments, 37749 sequences left"
```

```
## [1] "The following genes used for pooled deletion detection"
```

```
## [1] "IGHV3-23" "IGHV3-15" "IGHV1-18" "IGHV2-5" "IGHV3-53" "IGHV4-39"
```

```
## [7] "IGHV1-69" "IGHV3-7" "IGHV3-11" "IGHV3-49"
## [1] "5774 sequences were removed due to multiple assignments, 29869 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-23" "IGHV3-48" "IGHV5-51" "IGHV1-18" "IGHV3-49" "IGHV1-46"
## [7] "IGHV3-73"
## [1] "5374 sequences were removed due to multiple assignments, 34009 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV1-18" "IGHV1-46" "IGHV3-64D" "IGHV3-49" "IGHV2-5"
## [1] "5884 sequences were removed due to multiple assignments, 29681 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-30" "IGHV3-48" "IGHV3-7" "IGHV3-11" "IGHV3-49" "IGHV1-46"
## [1] "3954 sequences were removed due to multiple assignments, 28840 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-9" "IGHV1-69" "IGHV3-73"
## [1] "8991 sequences were removed due to multiple assignments, 44838 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-30" "IGHV3-48" "IGHV3-49" "IGHV4-59" "IGHV5-10-1"
## [6] "IGHV5-51" "IGHV3-53" "IGHV3-11" "IGHV1-69" "IGHV2-70"
## [11] "IGHV3-73" "IGHV1-58"
## [1] "5736 sequences were removed due to multiple assignments, 40847 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-11" "IGHV4-39" "IGHV1-46" "IGHV5-51" "IGHV3-48" "IGHV3-13"
## [7] "IGHV3-49" "IGHV3-73"
## [1] "5072 sequences were removed due to multiple assignments, 33204 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-23" "IGHV3-48" "IGHV1-46" "IGHV3-49" "IGHV3-7" "IGHV3-11"
## [7] "IGHV1-58" "IGHV3-13"
## [1] "1212 sequences were removed due to multiple assignments, 8737 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-53" "IGHV3-48" "IGHV3-11" "IGHV1-46" "IGHV1-69" "IGHV2-5"
## [7] "IGHV3-49"
## [1] "3624 sequences were removed due to multiple assignments, 22412 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-33" "IGHV3-30" "IGHV3-15" "IGHV1-46" "IGHV3-48"
## [6] "IGHV3-73" "IGHV3-49" "IGHV2-70" "IGHV5-10-1"
```

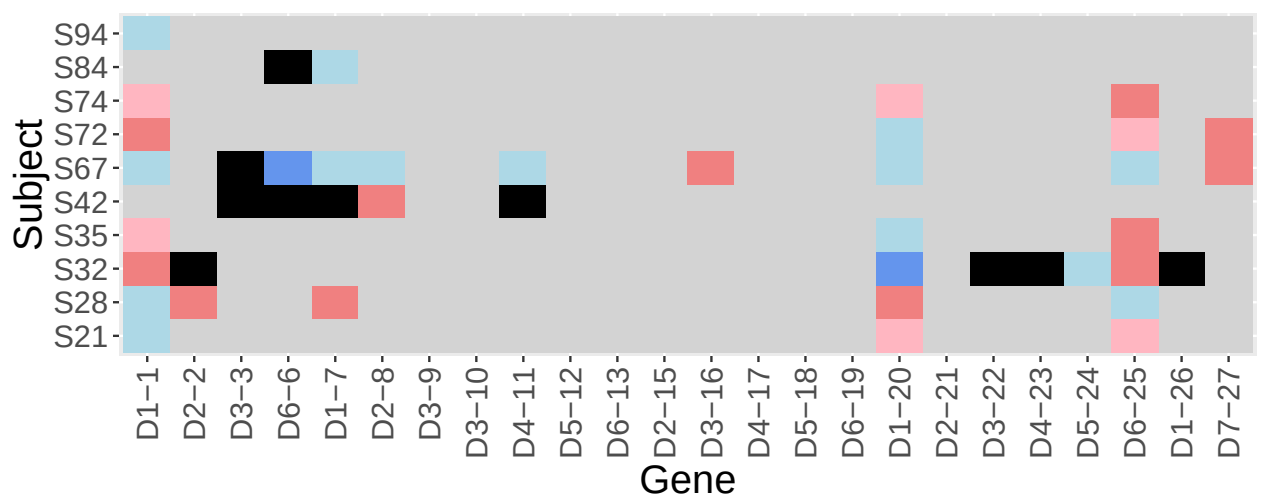
```
head(del_db)
```

	GENE	DELETION	K1_NEW	COUNTS1_NEW	V_GENE	K1	SUBJECT
## 1	IGHD1-1	0	19.08503	15,22	V(pooled)	19.09	S42
## 2	IGHD1-20	0	12.03093	10,17	V(pooled)	12.03	S42
## 3	IGHD1-26	0	39.61305	330,564	V(pooled)	39.61	S42
## 4	IGHD1-7	1	8.22609	7,68	V(pooled)	8.23	S42
## 5	IGHD2-15	0	198.5908	170,309	V(pooled)	198.59	S42
## 6	IGHD2-2	0	279.3872	234,405	V(pooled)	279.39	S42

For visualizing the deletion detected by `deletionsByVpooled`, the `plotDeletionsByVpooled` can be used. However, it is recommended to use this function for multiple individuals.

```
# Plot the deletion heatmap
```

```
plotDeletionsByVpooled(del_db)
```



Infering double chromosome deletion by relative gene usage

....

The `deletionsByBinom` function uses the V pooled approach to detect single chromosomal deletion for D and J.

```
# Inferred deletion summary table
del_binom_db <- deletionsByBinom(samples_db)
head(del_binom_db)

## # A tibble: 6 x 10
## # Groups:   foradj [1]
##   SUBJECT GENE      FRAC NREADS min_frac NREADS_SAMP pval foradj pval_adj
##   <chr>   <chr>   <dbl> <int>   <dbl>      <int> <dbl> <chr>   <dbl>
## 1 S42     IGHV~ 0.00714 42396 0.00471    42842 1 IGHV3~ 1
## 2 S94     IGHV~ 0.0152 34816 0.00471    35643 1 IGHV3~ 1
## 3 S72     IGHV~ 0.0105 38884 0.00471    39383 1 IGHV3~ 1
## 4 S21     IGHV~ 0.00582 35255 0.00471    35565 1 IGHV3~ 1
## 5 S28     IGHV~ 0.00793 32558 0.00471    32794 1 IGHV3~ 1
## 6 S84     IGHV~ 0.00855 53205 0.00471    53829 1 IGHV3~ 1
## # ... with 1 more variable: col <fct>
```

References

1. Gidoni *et al.* (2018).
2. Alamyar *et al.* (2010)
3. Munshaw and Kepler (2010)

4. Lefranc *et al.* (2003)